

INSTITUTO FEDERAL

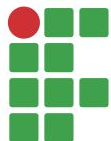
Mato Grosso do Sul

Construção de Páginas Web III

Milena Alegre

Checklist de Conteúdos

- ☐ 1. Manipulação de Arquivos
- ☐ 2. Sessões e Cookies
- ☐ 3. Geração de Relatórios
- ☐ 4. Criação de Aplicação Web Completa (S.O.L.I.D)
- ☐ 5. Filtro de Acesso



INSTITUTO FEDERAL

Mato Grosso do Sul

Manipulação de Arquivos

Dominando I/O no servidor com Node.js, Módulo File System (fs), Módulo path, Async/Await e Boas Práticas.

🎯 Objetivos da Aula

- ✓ Compreender a arquitetura de I/O em sistemas servidor.
- ✓ Dominar o módulo nativo `node:fs` e `node:fs/promises`.
- ✓ Manipular caminhos de arquivos cross-platform com `node:path`.
- ✓ Executar operações CRUD completas em arquivos de texto e JSON.
- ✓ Prevenir travamentos da Event Loop tratando exceções com `try...catch`.

☰ Estrutura do Conteúdo

1. **Conceitos Fundamentais:** Módulo `fs`, Sync vs Async.
2. **Importância Prática:** Logs, Configurações e Uploads.
3. **Módulo `path`:** Tratamento de rotas e separadores de SO.
4. **Operações CRUD:** Read, Write, Append, Unlink.
5. **Boas Práticas:** Exceções `ENOENT` e desativação de chamadas síncronas.
6. **Mão na Massa:** Projeto prático com persistência JSON.

Modo File System (fs)

O módulo nativo **File System (fs)** do Node.js fornece uma API completa para interagir com o sistema de arquivos do sistema operacional subjacente.



Módulo Built-in

Não exige instalação de pacotes via **npm**. Já vem integrado ao ecossistema do Node.js por padrão.

Com o fs, uma aplicação web pode ler arquivos do disco, criar relatórios em tempo de execução, persistir estados e gerenciar diretórios.

Modo File System (fs)

```
fs >  server.js > ...  
1   const fs = require("fs");  
2  
3   fs.writeFileSync("texto.txt", "Olá, mundo!");  
4  
5   console.log("Arquivo criado!");  
6  
7   |
```

pra testar rode no terminal: node nomeDoArquivo.js

O mais importante é: **não existe npm install fs**. Ele é um **módulo nativo (built-in)** do Node.js.

Modo File System (fs)

```
fs >  server.js > ...  
1  const fs = require("fs");  
2  // fs.writeFileSync("texto.txt", "Olá, mundo!");  
3  // console.log("Arquivo criado!");  
4  
5  //ler  
6  const texto = fs.readFileSync("texto.txt", "utf8");  
7  console.log(texto);
```

Síncrono x Assíncrono

🔒 Operações Síncronas (Sync)

Bloqueiam a linha de execução (Thread principal) até que a leitura ou escrita do arquivo seja finalizada pelo disco.

Atenção: Paralisa o atendimento de requisições web de outros usuários enquanto o disco responde.

⚡ Operações Assíncronas (Non-Blocking)

Delegam a tarefa de I/O para o sistema operacional através da biblioteca **libuv** e liberam a Event Loop imediatamente.

Recomendado: Essencial para servidores de alta performance e aplicações web concorrentes.

Regra de Ouro em Aplicações Web Node.js:

Nunca utilize funções síncronas (**readFileSync**, **writeFileSync**) dentro de rotas HTTP ativas! Use sempre a versão assíncrona baseada em Promises (**async/await**).

Callbacks, Promises, Sync

SÍNCRONO

1. Versão Sync

Simple, porém bloqueante. Lança exceções imediatamente.

```
server.js > ...  
const fs = require('node:fs');  
const data = fs.readFileSync('texto2.txt');
```

ASSÍNCRONO LEGADO

2. Callbacks (Err-first)

Usa funções de retorno. Pode causar Callback Hell.

```
server.js > ...  
const fs = require('node:fs');  
fs.readFile('texto2.txt', (err, data) => {  
  if (err) {  
    console.error('Erro ao ler o arquivo', err);  
    return;  
  }  
});
```

PADRÃO MODERNO

3. Promises (async/await)

Código limpo, legível e de fácil tratamento com try...catch.

```
server.js > ...  
const fs = require('node:fs/promises');  
  
async function lerArquivo() {  
  try {  
    const data = await fs.readFile('texto2.txt', 'utf8');  
    console.log('Conteúdo do arquivo:', data);  
  } catch (err) {  
    console.error('Ops, deu um erro ao ler o arquivo', err.message);  
  }  
}
```

📌 Nesta disciplina, focaremos **100% no padrão de Promises** com async/await via node:fs/promises.

Callbacks, Promises, Sync

PADRÃO MODERNO

3. Promises (async/await) Código limpo, legível e de fácil tratamento com try...catch.

```
server.js > ...  
const fs = require('node:fs/promises');  
  
async function lerMeuArquivo() {  
  try {  
    const data = await fs.readFile('texto2.txt', 'utf8');  
    console.log("Conteúdo do arquivo:", data);  
  } catch (erro) {  
    console.error("Ops, deu um erro ao ler o arquivo:", erro.message);  
  }  
}  
  
lerMeuArquivo();
```

Uso de Promises

- **Não trava o seu servidor:** Diferente do modo síncrono (como o `readFileSync` do primeiro exemplo), ele faz a leitura ou escrita do arquivo "nos bastidores". Isso significa que, se o seu programa for um site, ele pode continuar respondendo a outros usuários normalmente enquanto o arquivo carrega.
- **Código limpo com `async/await`:** Ele permite que você escreva operações assíncronas de uma forma muito natural, quase como se estivesse escrevendo uma receita de bolo, lendo uma linha após a outra.
- **Fuga do "Callback Hell":** No módulo `fs` antigo, se você quisesse ler um arquivo, alterar o texto e salvar de novo, precisaria colocar uma função dentro da outra. Isso criava um código cheio de recuos em formato de pirâmide, super confuso de ler e manter.
- **Tratamento de erros simplificado:** Ao usar promises, você pode envolver tudo em um bloco `try...catch`. Se qualquer coisa der errado no meio do caminho (como o arquivo não existir e gerar aquele erro `ENOENT`), o código pula direto para o `catch` de forma segura.

ENOENT = Error NO ENTry

Importância no Desenvolvimento

Por que um desenvolvedor web backend precisa dominar a manipulação de arquivos?

Autonomia no Servidor

Nem todos os dados de um sistema justificam a complexidade de um banco de dados relacional ou NoSQL. Arquivos locais resolvem problemas de armazenamento rápido, temporário e direto.

Integração de Sistemas

Exportação de dados (relatórios CSV, relatórios PDF), importação de planilhas de clientes e comunicação entre processos via arquivos compartilhados.

Casos de Uso: Logs de Sistema

Registro de Logs e Auditoria

Aplicações web corporativas registram erros, acessos e eventos críticos em arquivos físicos de log (ex: access.log, **error.log**).

- ✓ Rastreamento de falhas em produção.
- ✓ Registro temporal de ações de usuários (auditoria).
- ✓ Análise posterior por ferramentas de monitoramento.

Casos de Uso: Configuração

Arquivos de Configuração

Arquivos como `package.json`, `config.json` ou arquivos de ambiente são lidos durante a inicialização do servidor.

Vantagem: Alterar portas, chaves de API ou parâmetros do sistema sem precisar recompilar ou alterar o código-fonte principal da aplicação.

Casos de Uso: Uploads e Mídias

Upload de Imagens

Recepção de avatares, fotos de produtos e documentos enviados por formulários web.

Geradores de PDFs

Criação dinâmica de comprovantes, relatórios financeiros e boletos para download do cliente.

Persistência Leve

Prototipagem de APIs e microservices armazenando dados temporários em arquivos JSON.

 **Dica Prática:** Em ambientes de produção reais, arquivos de mídias pesados costumam ser enviados para Storage Servers (como AWS S3), enquanto arquivos de apoio local continuam sendo manipulados diretamente no servidor pelo Node.js.

O módulo nativo `node:path` do Node.js é uma ferramenta essencial para lidar com caminhos de arquivos e diretórios. Ele existe principalmente porque diferentes sistemas operacionais usam barras diferentes para separar pastas (o Windows usa contrabarra `\`, enquanto o Mac e o Linux usam barra normal `/`).

O módulo `path` padroniza isso automaticamente para o seu código funcionar em qualquer computador.

A função `path.join()` junta todos os segmentos fornecidos usando o separador específico do SO onde o Node.js está rodando.

- ✓ Normaliza barras duplicadas automaticamente.
- ✓ Resolve segmentos relativos (`..` e `.`).
- ✓ Evita erros ao migrar do Windows para servidores Linux em nuvem.

O método `path.resolve()` tem um objetivo diferente: ele tenta descobrir o **caminho absoluto** de um arquivo ou pasta. Um caminho absoluto é aquele endereço completo que começa na raiz do seu disco rígido (por exemplo, `/Users/milenaalegre/...`).

- Ele avalia os parâmetros da direita para a esquerda, tentando montar o caminho raiz.
- Se os parâmetros não formarem um caminho absoluto por si só, ele automaticamente anexa o caminho da pasta atual onde o seu terminal está executando o Node.js.

`path.join()`

Concatena (junta) segmentos de caminho com a barra correta.

Relativo (depende apenas dos textos que você passou).

`path.resolve()`

Descobre o endereço real e completo no sistema.

Absoluto (desde a raiz do disco rígido).

Mapeamento: ESM x Commons JS

CommonJS (CJS): É o padrão tradicional e mais antigo do Node.js. Ele carrega os arquivos de forma síncrona (bloqueante) e foi criado antes do JavaScript ter um sistema de módulos oficial.

ES Modules (ESM): É o padrão moderno e oficial da linguagem JavaScript. Ele carrega os arquivos de forma assíncrona, sendo o padrão utilizado nos navegadores modernos e o formato recomendado para novos projetos no Node.js.

A maior diferença no dia a dia é a sintaxe de como você importa e exporta coisas entre os seus arquivos. Aqui está uma tabela de mapeamento prático:

Ação	CommonJS (Padrão Tradicional)	ES Modules (Padrão Moderno)
Importar um módulo	<code>const fs = require('node:fs');</code>	<code>import fs from 'node:fs';</code>
Exportar algo principal	<code>module.exports = minhaFuncao;</code>	<code>export default minhaFuncao;</code>
Importar funções específicas	<code>const { soma } = require('./math');</code>	<code>import { soma } from './math.js';</code>
Exportar funções específicas	<code>exports.soma = soma;</code>	<code>export const soma = (a, b) => a+b;</code>

Operações Essenciais (C.R.U.D)

O ciclo de vida básico de manipulação de dados em arquivos corresponde às 4 operações CRUD:



CREATE

`fs.writeFile()`

cria novo arquivo



READ

`fs.readFile()`

lê do disco



UPDATE

`fs.appendFile()`

acrescenta dados



DELETE

`fs.unlink()`

exclui arquivo

Leitura com fs.readFile

Lendo Conteúdo de Arquivos

O método `fs.readFile(caminho, encoding)` realiza a leitura assíncrona.

Atenção ao Encoding!

Se você omitir o segundo parâmetro ('utf-8'), a função devolverá um objeto binário **Buffer** em vez de uma string legível!

```
const data = await fs.readFile('a.txt', 'utf8');
```

Entendendo Encodings e Buffers

O que é um Buffer?

Um **Buffer** representa uma sequência de bytes brutos na memória RAM do computador (ex:).

É a estrutura ideal para manipular arquivos binários, como imagens PNG/JPG, vídeos, áudios e PDFs.

O Encoding UTF-8

O encoding traduz os bytes brutos do Buffer em caracteres de texto compreensíveis por seres humanos (letras, números, acentos e emojis).

Você também pode converter um buffer para string manualmente via `buffer.toString('utf-8')`.

Escrita fs.writeFile

Criando e Sobrecrevendo

O método `fs.writeFile(caminho, dados, encoding)` cria um novo arquivo no disco.

Comportamento Destrutivo!

Se o arquivo já existir no caminho especificado, o seu conteúdo anterior será **completamente apagado e substituído** sem aviso prévio.

Inclusão com fs.appendFile

Adicionando Dados ao Final

Quando precisamos preservar o histórico existente (como em registros de log ou históricos de conversas), utilizamos `fs.appendFile()`.

- ✓ Se o arquivo não existir, ele será criado.
- ✓ Se o arquivo já existir, o novo texto é anexado ao fim.
- ✓ Lembre-se de incluir a quebra de linha `\n`.

```
import fs from 'node:fs/promises';  
fs.appendFile("texto2.pdf", "\n blablabla");  
console.log("Arquivo criado!");
```

Exclusão com fs.unlink

Removendo Arquivos do Disco

A função `fs.unlink(caminho)` apaga definitivamente um arquivo individual do sistema de arquivos.

Atenção ao apagar!

O arquivo não vai para a Lixeira do sistema operacional. A exclusão é imediata e irreversível!

```
import fs from 'node:fs/promises';  
fs.unlink("texto3.txt");  
console.log("Arquivo destruido!");
```

Comparação

Operação	Método (Promises)	Ação no Arquivo	Se o arquivo NÃO existir?
READ	<code>fs.readFile()</code>	Lê o conteúdo do disco	Lança Erro (ENOENT)
CREATE / WRITE	<code>fs.writeFile()</code>	Substitui todo o conteúdo	Cria novo arquivo
UPDATE / APPEND	<code>fs.appendFile()</code>	Anexa dados ao final	Cria novo arquivo
DELETE	<code>fs.unlink()</code>	Remove o arquivo do SO	Lança Erro (ENOENT)

Além de arquivos individuais, o módulo fs permite criar, listar e remover pastas no servidor:

fs.mkdir(caminho, { recursive: true }): Cria pasta (inclusive pastas pai se não existirem).

fs.readdir(caminho): Lista o nome de todos os arquivos contidos no diretório.

fs.rm(caminho, { recursive: true }): Remove uma pasta e todo o seu conteúdo interno.

Persistência e Performance

O armazenamento em disco rígido (SSD/HDD) é ordens de grandeza mais lento do que operações na memória RAM.

A arquitetura orientada a eventos do Node.js permite gerenciar mil conexões simultâneas enquanto grava informações no disco em segundo plano.

🔑 Lendo e Convertendo com `JSON.parse()`

Como os arquivos armazenam apenas texto plano, precisamos converter a string lida em objetos/arrays manipuláveis do JavaScript.

📦 Formatando com `JSON.stringify()`

Para salvar de volta no arquivo, convertemos o objeto JS em texto usando `JSON.stringify(dados, null, 2)`.

O parâmetro `null, 2` formata o JSON com indentação de 2 espaços, mantendo o arquivo legível para humanos.

🔗 Código Completo e Seguro

Unindo a resolução de caminhos com `node:path`, leitura assíncrona com `async/await` e tratamento de exceções.

```
server.js > ...
import fs from 'node:fs/promises';
import path from 'node:path';

try {
  // 1. Construção segura do caminho do arquivo
  const caminhoDoArquivo = path.join(process.cwd(), 'pasta', 'arquivo.txt');

  // 2. Leitura assíncrona aguardada com await e texto formatado
  const conteudo = await fs.readFile(caminhoDoArquivo, 'utf8');

  console.log("Conteúdo lido com sucesso:\n", conteudo);
} catch (erro) {
  // 3. Captura e tratamento de qualquer erro de execução
  console.error("Erro ao tentar ler o arquivo:", erro.message);
}
```

Middleware no Express

O que é um Middleware?

É uma função intermediária no Express que intercepta a requisição (req) antes que ela chegue ao seu destino final ou envie uma resposta (res).

Ela tem acesso aos dados da requisição e pode alterá-los, validar parâmetros ou interromper a execução.

Para que serve?

Serve para separar responsabilidades na aplicação, como por exemplo:

- Converter o corpo da requisição (JSON ou Formulário).
- Autenticar se o usuário está logado.
- Registrar logs de acesso no terminal.

Entendendo `app.use(express.urlencoded())`



1. Interceptação Global

O comando `app.use(...)` registra um middleware global. Isso significa que ele será executado para **todas** as rotas do seu servidor.



2. Parse de Formulário

O `express.urlencoded()` pega os dados brutos enviados por formulários HTML () e faz o decodificação da URL.



3. Criação do `req.body`

Ele transforma o texto recebido em um objeto JavaScript acessível. É graças a ele que podemos usar `req.body.nome` e `req.body.mensagem`.

Conceito Inicial

```
server.js > ...  
import express from 'express';  
const app = express();  
  
// Habilita a leitura de formulários HTML  
app.use(express.urlencoded({ extended: true }));  
  
app.post('/cadastro', (req, res) => {  
  // Acessa os campos do formulário diretamente do req.body  
  console.log(req.body.nome);  
  console.log(req.body.idade);  
});
```

Por que especificamos 'utf-8'?

Codificação de Caracteres

O **UTF-8** é o padrão universal de codificação. Ele garante que acentos (ã, é, ó), cedilhas (ç) e caracteres especiais sejam interpretados corretamente.

Aplicação no Módulo FS

Ao ler ou gravar com `fs.readFileSync('mural.txt', 'utf-8')`, dizemos ao Node para retornar uma **string de texto legível** em vez de um buffer de bytes brutos.

O Método `.map()` em Arrays

 **Transformação de Listas:** O método `.map()` percorre cada item de um Array e constrói um novo Array com os elementos modificados.

 **Conversão para HTML:** No nosso código, usamos `linhas.map(linha => `${linha}`)` para transformar cada linha do texto em um elemento de lista.

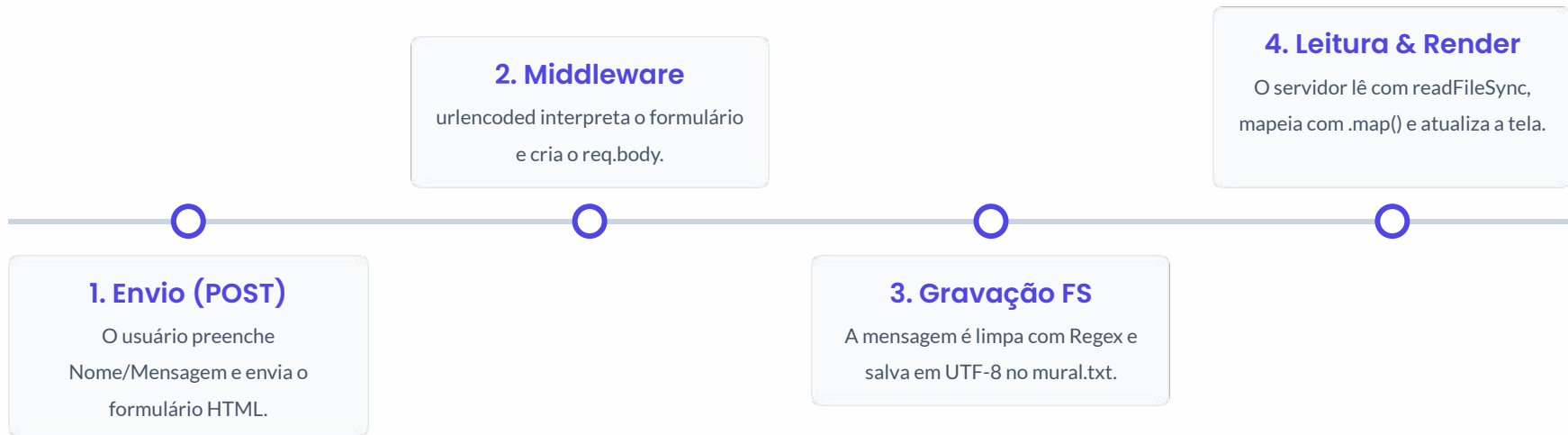
 **Junção com `.join()`:** Em seguida, o `.join("")` une todos os itens em uma única string HTML para exibir no navegador.

Conceito Inicial

```
const numeros = [2, 4, 6];  
const dobrados = numeros.map(num => num * 2);  
  
console.log(dobrados); // [4, 8, 12]  
console.log(numeros);  // [2, 4, 6] (Original continua igual)
```

```
const alunos = [  
  { id: 1, nome: 'Milena', nota: 9 },  
  { id: 2, nome: 'Jose', nota: 8 }  
];  
  
// Pega apenas os nomes dos alunos e cria um array de strings  
const nomes = alunos.map(aluno => aluno.nome);  
  
console.log(nomes); // ['Milena', 'Jose']
```

Fluxo Completo de Execução



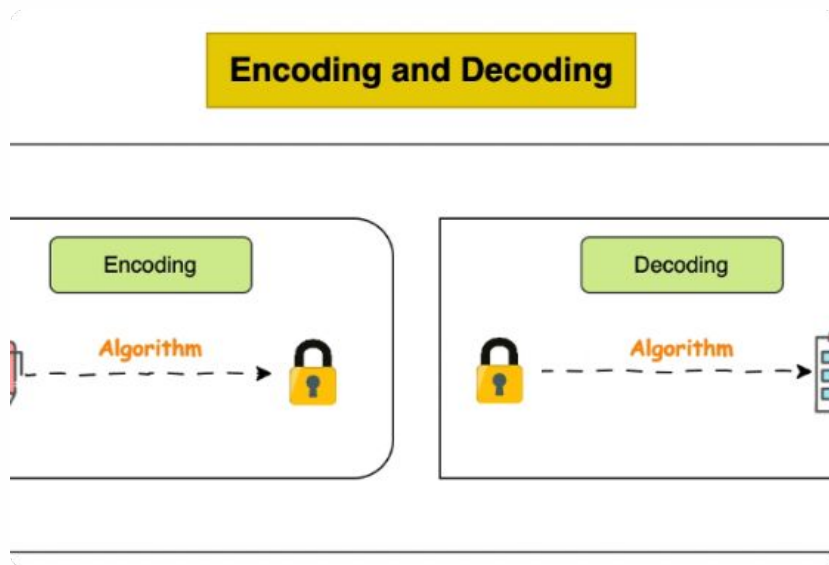
Organização e Boas Práticas

Mantenha middlewares estruturados no topo do código antes das rotas.

Sempre defina o encoding utf-8 na manipulação de arquivos de texto.

Trate exceções e validações do req.body para evitar salvar dados vazios.

O uso de arquivos em texto é ótimo para estudos, enquanto aplicações em produção utilizam Bancos de Dados.



Conceito Inicial



Fabricar Documentos

Gerar e exportar relatórios em PDF,
planilhas organizadas e extratos
customizados

Upload Seguro de Arquivos



Para fazer upload de arquivos no Express (como foto de perfil), o `express.urlencoded()` **não funciona**, pois ele só lê texto. Você precisa usar o middleware **multer**, que é a biblioteca padrão do Node.js para processar envio de arquivos.

O HTML (Formulário)

O ponto mais importante no HTML é adicionar o atributo `enctype="multipart/form-data"` no formulário. Sem ele, a foto não é enviada para o servidor.

```
npm install multer
```

Pergunta...

Isso é um banco de dados?

Quais limitações em comparação a um banco de dados real?

Na prática do mercado, imagens e arquivos físicos ficam salvos no disco ou em um serviço de armazenamento (como AWS S3 ou Cloudinary), enquanto **apenas o caminho em texto** (ex: [uploads/171198.png](#)) é gravado em um banco de dados real (como PostgreSQL, MySQL ou MongoDB) associado ao cadastro do usuário.

Limitações do Sistema de Arquivos vs. Banco de Dados Real

- **Sem capacidade de consulta (Queries).**
- **Falta relacionamento entre dados.**
- **Perda de dados em servidores (Escalabilidade):** Hospedagens modernas (como Vercel, Heroku ou Render) limpam o disco local toda vez que o código é atualizado ou reiniciado. As fotos salvas na pasta [uploads/](#) sumiriam.
- **Acesso Concorrente e Corrupção:** Se dezenas de usuários tentarem enviar, sobrescrever ou ler arquivos ao mesmo tempo no mesmo disco, a aplicação pode travar ou corromper dados. Bancos reais lidam com **transações ACID** para garantir que nada se quebre.
- **Segurança e Permissões**

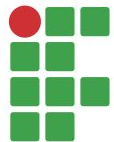
>_ Desafio: Mural de Recados

-  **Passo 1:** Criar um formulário HTML simples com os campos Nome e Mensagem.
-  **Passo 2:** Criar a rota POST no seu servidor Node/Express para receber os dados informados.
-  **Passo 3:** Gravar cada recado em uma nova linha do arquivo mural.txt usando fs.appendFileSync.
-  **Passo Bônus:** Criar uma página que leia o arquivo mural.txt e renderize todas as mensagens salvas diretamente na tela.

>_ Desafio: Diário de Estudo

- ✓ **Passo 1:** Crie um formulário HTML com os campos **Matéria** (`<input>`) e **O que você aprendeu hoje?** (`<textarea>`). O formulário deve enviar os dados via `POST` para a rota `/estudo`.
- ✓ **Passo 2:** No seu servidor Node/Express, crie a rota `POST /estudo` para capturar os dados do formulário (`req.body`).
- ✓ **Passo 3:** Grave cada anotação em uma nova linha do arquivo `diario.txt` usando o método `fs.appendFileSync`. O texto salvo deve seguir a estrutura: `[MATÉRIA]: Resumo do aprendizado`
- ★ **Passo Bônus:** Crie a rota `GET /historico` que leia o arquivo `diario.txt` usando `fs.readFileSync` e exiba todas as anotações salvas diretamente na tela do navegador e adicione estilo.

milena.alegre@ifms.edu.br



INSTITUTO FEDERAL
Mato Grosso do Sul